DESENVOLVIMENTO MOBILE · MÓDULO 00 · COMUM ÀS TRILHAS · NÍVEL INICIANTE

Apresentação e Ecossistema Mobile

O curso e suas duas trilhas, história da computação móvel, arquitetura do Android e do iOS, ciclo de vida dos apps e abordagens de desenvolvimento

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Entender a organização do curso, suas duas trilhas (Kotlin e Flutter) e o projeto LabMonitor.
- Explicar o que diferencia o desenvolvimento mobile do web e do desktop.
- Contar a evolução da computação móvel e situar as tecnologias atuais.
- Descrever a arquitetura em camadas do Android e do iOS e o ciclo de vida de um app.
- Comparar as abordagens nativa, multiplataforma, híbrida e PWA.
- Escolher a trilha mais adequada ao seu objetivo.

Pré-requisitos

Lógica de programação e noções de orientação a objetos. Experiência com Java, C, C++ ou Python ajuda, mas não é obrigatória.

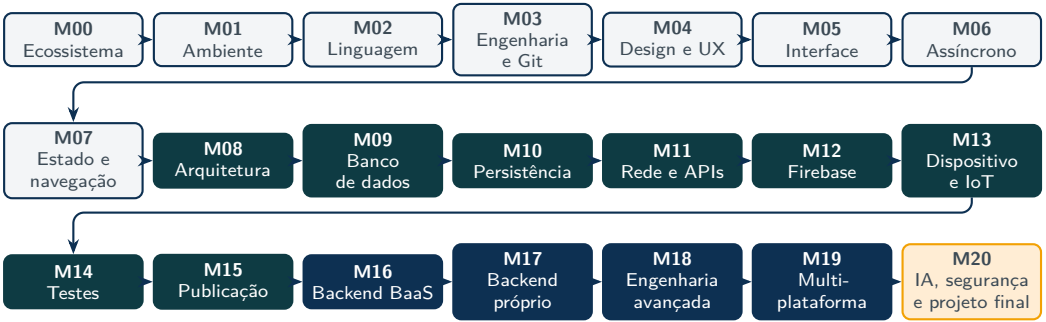
Roteiro

Neste módulo

- Sobre o curso
- Fundamentos: o que é computação móvel
- Plataformas: Android e iOS
- Abordagens e tecnologias
- Fechamento

Sobre o curso

Do iniciante ao avançado em 21 módulos



Iniciante (M00–M07): fundamentos, linguagem, interface e estado.

Intermediário (M08–M15): arquitetura, dados, rede, hardware, testes e publicação.

Avançado (M16–M20): backend, engenharia em escala, multiplataforma, IA e projeto final.

Duas trilhas, a mesma numeração

Módulo	Trilha Kotlin (Android nativo)	Trilha Flutter (Android, iOS, web, desktop)
01	Android Studio, Gradle e emulador	Flutter SDK, DevTools e emuladores
02	Linguagem Kotlin	Linguagem Dart
05	Jetpack Compose	Widgets e layouts
06	*Coroutines* e `Flow`	`Future`, `Stream` e *isolates*
07	Estado e Navigation 3	Estado, `go_router` e animações
08	MVVM com Hilt	MVVM com Provider, Riverpod e Bloc
10	Room e DataStore	Drift, sqflite e `shared_preferences`
11	Retrofit e OkHttp	`http` e Dio
12	Firebase	FlutterFire
13	Dispositivo e IoT	Dispositivo completo e IoT
14	Testes (JUnit, Compose)	Testes (unidade, widget, *golden*, integração)
15	Google Play	Google Play e App Store
19	Multiplataforma (KMP, Flutter, RN)	Código nativo, web, desktop e embarcados
20	IA, segurança e projeto final	IA, segurança e projeto final

Módulos **comuns** às duas trilhas: 00, 03, 04, 09, 16, 17 e 18.

Como escolher a sua trilha

Trilha Kotlin

Para quem quer ser desenvolvedor **Android**, trabalhar com o ecossistema oficial do Google (Jetpack) e depois expandir para iOS com Kotlin Multiplatform.

Trilha Flutter

Para quem precisa entregar **Android e iOS** (e web e desktop) com um único código e uma interface idêntica em todas as plataformas.

Se você...	Comece por
quer vaga de Android nativo	Kotlin
precisa publicar também na App Store	Flutter
não tem Mac, mas quer iOS no futuro	Kotlin ou Flutter
quer painel web/desktop do mesmo app	Flutter
já programa em Java	Kotlin

Dica

Os conceitos (estado, arquitetura, testes, dados) se transferem: quem conclui uma trilha aprende a outra em poucas semanas.

Como ler estes slides

Definição

Significado preciso de um termo.

Conceito

Ideia central explicada.

Por dentro

Como a tecnologia funciona por baixo.

Exemplo

Caso concreto, quase sempre do LabMonitor.

Analogia

Comparação com o dia a dia.

Um pouco de história

Contexto: de onde veio a ideia.

Dica

Atalho ou boa prática.

Atenção

Cuidado: regra, limite ou armadilha.

Erro comum

Engano frequente e como evitá-lo.

Exercício

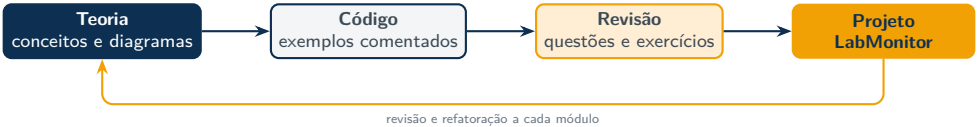
Prática para fixar.

Projeto LabMonitor

Etapa do projeto integrador.

Cada módulo: **fundamentos** → **prática** → resumo, glossário, questões com respostas, exercícios.

Metodologia de estudo



Como estudar

- Leia os fundamentos antes do código.
- Responda às questões de revisão **antes** de olhar as respostas.
- Digite o código, não copie e cole; rode no emulador *e* num aparelho real.
- Faça um *commit* ao fim de cada etapa.

Carga sugerida

- 8 a 12 horas por módulo, incluindo exercícios.
- Os módulos 02, 05, 06, 07 e 08 são a base de tudo: não pule.
- Avançados (16–20) supõem os intermediários concluídos.

Projeto integrador: *LabMonitor*



Um app para **laboratórios de eletroeletrônica**, construído ao longo do curso:

- requisitos, protótipo no Figma e repositório (**M03–M04**);
- bancadas, equipamentos e reservas com dados locais (**M05–M10**);
- sincronização com a nuvem e backend (**M11–M12**, **M16–M17**);
- sensores de um **ESP32** via BLE e MQTT (**M13**);
- testes, publicação e CI/CD (**M14–M15**, **M18**);
- detecção de anomalias e assistente com IA (**M20**).

Por que um projeto integrador?

Cada conceito novo é aplicado no mesmo código, como acontece num produto real que evolui.

Fundamentos: o que é computação móvel

O ambiente móvel é diferente

Computação móvel

Uso de dispositivos portáteis, com bateria e conexão sem fio, que acompanham a pessoa e percebem o ambiente por sensores.

- **Recursos limitados**: bateria, memória e CPU compartilhados com dezenas de apps.
- **Rede intermitente**: 4G/5G, Wi-Fi e modo avião; o app precisa funcionar *offline*.
- **Telas diversas**: de relógios a tablets e dobráveis.
- **Interrupções**: chamadas, notificações e troca de app a qualquer momento.
- **Ciclo de vida controlado pelo sistema**: o processo pode ser encerrado para liberar memória.
- **Sensores**: câmera, GPS, acelerômetro, Bluetooth, NFC.
- **Privacidade**: permissões explícitas e isolamento por app.
- **Distribuição por lojas**: revisão, políticas e atualizações graduais.

Consequência prática

Um bom app é projetado para **ser interrompido**, **perder a rede** e **ser encerrado** sem perder dados.

Antes do smartphone



Um pouco de história

Em 1973, Martin Cooper, da Motorola, fez a primeira ligação de um telefone celular portátil, em Nova York. Os aparelhos da época pesavam cerca de um quilo.

O que faltava

Telas pequenas e resistivas, internet lenta e cara, e nenhum lugar central para encontrar e instalar programas.

A era dos apps



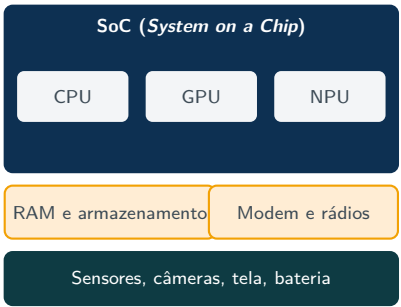
A virada de 2007–2008

Tela multitoque grande, navegador completo e, principalmente, **lojas de apps**: qualquer desenvolvedor passou a distribuir software para milhões de pessoas.

Tendência

As interfaces migraram do modelo **imperativo** (XML, UIKit) para o **declarativo** (Compose, SwiftUI, Flutter). Materiais antigos ainda usam o modelo anterior: saiba reconhecê-lo, mas comece pelo atual.

Dentro do aparelho



SoC

Um único chip que reúne processador (CPU), processador gráfico (GPU), acelerador de IA (NPU), modem e controladores. Economiza espaço e energia.

Por que isso importa para o dev

A GPU desenha a interface; a NPU acelera modelos de IA no aparelho (Módulo 20); o rádio é um dos maiores consumidores de bateria (Módulo 18).

ARM

Quase todos os celulares usam processadores com arquitetura ARM, projetados para alto desempenho por watt.

Plataformas: Android e iOS

Android *versus* iOS

	Android	iOS
Linguagem oficial	Kotlin (Java ainda suportado)	Swift (Objective-C legado)
UI moderna	Jetpack Compose	SwiftUI
IDE	Android Studio (Windows, Linux, macOS)	Xcode (somente macOS)
Código	núcleo aberto (AOSP), personalizado por fabricantes	fechado, controlado pela Apple
Loja	Google Play (e lojas alternativas)	App Store
Conta de desenvolvedor	taxa única	anuidade
Dispositivos	milhares de modelos e fabricantes	poucos modelos, da Apple
Desafio típico	fragmentação de versões e telas	revisão rigorosa e ecossistema fechado

No Brasil

O Android está na grande maioria dos smartphones em uso; o iOS concentra parte relevante do público de maior renda.

iOS sem Mac?

Escreve-se o código Flutter ou KMP em qualquer sistema, mas compilar para iOS exige macOS e Xcode (ou CI na nuvem).

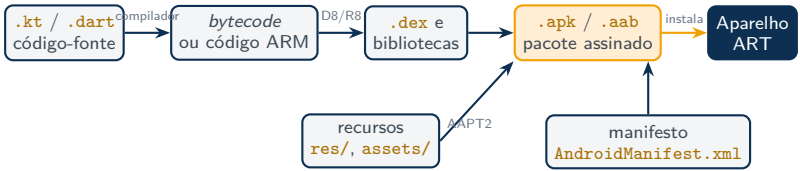
As pilhas de software



A mesma ideia: camadas

Embaixo, o núcleo do sistema (processos, memória, drivers); no meio, serviços e *runtime*; em cima, os *frameworks* que o app usa. Seu código fala quase sempre só com a camada de cima, e o Flutter acrescenta o próprio motor sobre ela (Módulo 19).

Do código-fonte ao aparelho (Android)



APK e AAB

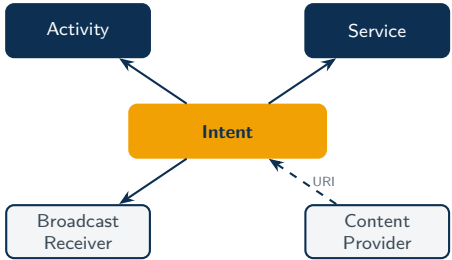
O **APK** é o arquivo instalável; o **AAB** é o formato de **envio** à Google Play, que gera APKs otimizados para cada aparelho (Módulo 15).

E no iOS?

O Xcode compila Swift (ou o Dart do Flutter) para código ARM e empacota num `.ipa`. Gradle e Xcode orquestram as etapas; o Flutter os comanda por você.

Componentes de um app Android

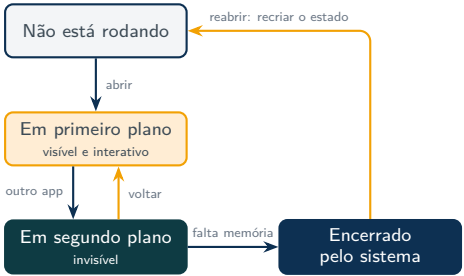
- **Activity**: uma tela com a qual o usuário interage. Apps em Compose ou Flutter costumam ter **uma única** Activity.
- **Service**: trabalho sem interface (tocar música, sincronizar).
- **BroadcastReceiver**: reage a eventos do sistema (bateria fraca, boot).
- **ContentProvider**: expõe dados a outros apps (contatos).
- **Intent**: mensagem que ativa componentes, do seu app ou de outros.



Dica

Todos os componentes e as permissões são declarados no `AndroidManifest.xml`, inclusive num app Flutter (pasta `android/`).

O ciclo de vida de um app



Ciclo de vida

Conjunto de estados por que o app passa e de eventos que o sistema envia nas transições (no Android, `onCreate`, `onStop`...; no iOS, *active*, *background*...).

Morte do processo

Em segundo plano, o sistema pode encerrar o app sem aviso. Se o estado da tela só existia na memória, o usuário volta e encontra tudo vazio.

Dica

Salve o essencial (rascunhos, posição) e reconstrua a partir do banco local (Módulos 07 e 10).

Níveis de API do Android

- **API level**: número que identifica cada versão do Android (Android 16 = API 36).
- `minSdk`: versão **mínima** em que o app instala.
- `targetSdk`: versão para a qual o app foi **testado**; ativa os comportamentos novos do sistema.
- `compileSdk`: versão das APIs usadas na **compilação**.

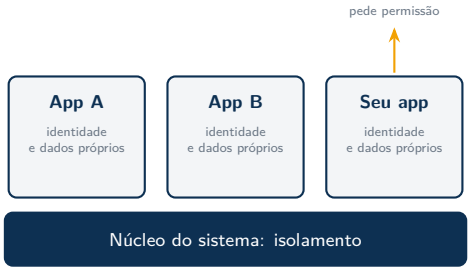
app/build.gradle.kts

```
android {
  compileSdk = 36
  defaultConfig {
    minSdk = 26      // Android 8.0
    targetSdk = 36   // Android 16
  }
}
```

Exigência da Google Play

Desde **31/08/2026**, apps novos e atualizações precisam de `targetSdk` 36 ou maior. O requisito sobe todo ano; no iOS, a Apple exige anualmente o SDK mais novo (Xcode 26 desde 28/04/2026).

Segurança: isolamento e permissões



Sandbox

Cada app roda isolado, com identidade própria; não lê os arquivos privados de outro app. É aplicação do **princípio do menor privilégio**.

- Permissões **normais** (internet) são concedidas na instalação.
- Permissões **perigosas** (câmera, localização) são pedidas **em tempo de execução**, e o usuário pode negar.

Verificação de desenvolvedor no Brasil

Desde **30/09/2026**, aparelhos Android certificados no Brasil só instalam normalmente apps de **desenvolvedores verificados**, mesmo fora da Play. Instalar via **ADB** durante o desenvolvimento continua funcionando (Módulos 01 e 15).

Abordagens e tecnologias

Abordagens de desenvolvimento

Abordagem	Como funciona	Vantagens	Limitações
Nativa	Kotlin (Android) ou Swift (iOS), com o SDK oficial	desempenho máximo, acesso total ao hardware	dois códigos para duas plataformas
Multiplataforma	Flutter, React Native, Kotlin Multiplatform	um código (ou a lógica) para Android e iOS	camada extra; recursos novos chegam depois
Híbrida	app web dentro de uma *WebView* (Capacitor, Ionic)	reaproveita HTML, CSS e JS	desempenho e aparência inferiores
PWA	site instalável com *service workers*	sem loja, atualização imediata	acesso limitado ao hardware, sobretudo no iOS

Dica

Não existe abordagem "melhor". Existe a mais adequada ao **público**, ao **orçamento**, à **equipe** e aos **recursos de hardware** exigidos. O Módulo 19 aprofunda a comparação.

As tecnologias do curso

Trilha Kotlin

- Kotlin, *coroutines*, `Flow`
- Jetpack Compose, Material 3
- ViewModel, Hilt
- Room, DataStore
- Retrofit, OkHttp

Trilha Flutter

- Dart, `Future`, `Stream`
- Widgets, Material 3, Cupertino
- MVVM, Provider (Riverpod, Bloc)
- Drift, sqflite, preferências
- `http`, Dio

Comum às duas

- Git e GitHub, Figma
- SQL e PostgreSQL
- Firebase e Supabase
- Dart (Shelf) ou Ktor no servidor, Docker
- GitHub Actions, ESP32 e MQTT

As ferramentas mudam; os fundamentos (estado, arquitetura, dados, redes, testes, segurança) permanecem. Por isso cada módulo começa pela teoria.

Quem faz um app: papéis na equipe

Papel	Responsabilidade	Módulos
Dono do produto (PO)	decide o que construir e em que ordem	03, 20
Designer de UX/UI	pesquisa, fluxos, protótipos e sistema de design	04
Desenvolvedor mobile	app: interface, estado, dados, integração com o hardware	01–15, 19
Desenvolvedor *backend*	APIs, banco de dados, autenticação	09, 16, 17
Qualidade (QA)	estratégia de testes e testes exploratórios	14
DevOps / plataforma	CI/CD, infraestrutura, observabilidade	17, 18

Dica

Em equipes pequenas, a mesma pessoa acumula vários papéis. Este curso passa por todos para que você entenda o trabalho de cada um e converse bem com eles.

Fechamento

Resumo

Resumo

- O curso tem 21 módulos em três níveis e duas trilhas (Kotlin e Flutter) com a mesma numeração.
- O ambiente móvel impõe restrições de energia, rede, tela, interrupções e ciclo de vida.
- Das primeiras ligações (1973) às lojas de apps (2008) e às interfaces declarativas (2019+).
- Android e iOS são pilhas em camadas; o app usa os *frameworks* do topo.
- O sistema pode encerrar o app em segundo plano: o estado precisa ser salvo e recriado.
- Nativo, multiplataforma, híbrido e PWA têm compromissos diferentes.

Glossário

Termo	Significado
Computação móvel	uso de dispositivos portáteis, com bateria e rede sem fio
SoC	chip único com CPU, GPU, NPU, modem e controladores
Kernel	núcleo do sistema operacional: processos, memória e drivers
HAL	camada que padroniza o acesso ao hardware de fabricantes diferentes
ART	*runtime* que executa o código dos apps Android
Ciclo de vida	estados e transições de um app controlados pelo sistema
Sandbox	isolamento que impede um app de acessar os dados de outro
API level	número de cada versão do Android (`minSdk`, `targetSdk`)
Abordagem nativa	app escrito com a linguagem e o SDK oficiais da plataforma

Questões de revisão

1. Cite quatro diferenças entre desenvolver para celular e para desktop.
2. O que as lojas de apps (2008) mudaram para quem desenvolve software?
3. Compare as pilhas do Android e do iOS: o que elas têm em comum?
4. Por que o sistema pode encerrar um app em segundo plano, e o que o desenvolvedor deve fazer a respeito?
5. Explique a diferença entre `minSdk` e `targetSdk`.
6. O que é o *sandbox* e qual princípio de segurança ele aplica?
7. Para um app que lê um sensor BLE e precisa estar no Android e no iOS em três meses, que abordagem você escolheria? Por quê?
8. Qual trilha do curso combina melhor com o seu objetivo profissional? Justifique.

Exercícios

Exercício 1 — Pesquisa

Escolha três apps que você usa todos os dias e tente identificar a abordagem de cada um (nativa, multiplataforma, híbrida ou PWA). Que pistas você usou?

Exercício 2 — Análise

Indique e justifique uma abordagem para: (a) app de lanterna; (b) app de um evento acadêmico de uma semana; (c) app que lê um sensor BLE a 100 Hz; (d) *startup* que precisa lançar em Android e iOS em três meses.

Exercício 3 — Ciclo de vida

Abra um app de formulário, preencha um campo, ative "Não manter atividades" nas opções de desenvolvedor do Android e volte ao app. O que aconteceu? Por quê?

LabMonitor — Etapa 0

Escreva em uma página: quem usa o app, quais as cinco funcionalidades principais, quais recursos de hardware ele usa e qual trilha você vai seguir.

Para aprofundar

- Visão geral da plataforma Android: `https://developer.android.com/guide/platform`
- Fundamentos de apps Android: `https://developer.android.com/guide/components/fundamentals`
- Ciclo de vida no iOS:
  `https://developer.apple.com/documentation/uikit/managing-your-app-s-life-cycle`
- Documentação do Flutter: `https://docs.flutter.dev`
- Curso oficial Android Basics with Compose:
  `https://developer.android.com/courses/android-basics-compose/course`
- Requisitos de API da Google Play:
  `https://developer.android.com/google/play/requirements/target-sdk`

Respostas das questões de revisão

1. Bateria e memória limitadas, rede intermitente, interrupções e encerramento pelo sistema, telas variadas, sensores, permissões e distribuição por lojas.
2. Criaram um canal central para distribuir e vender software a milhões de pessoas, com instalação e atualização simples; abriram o mercado para desenvolvedores independentes.
3. Ambas têm camadas: núcleo do sistema embaixo, serviços e *runtime* no meio e *frameworks* de interface em cima; o app fala com a camada superior.
4. Para liberar memória para o app em primeiro plano. O desenvolvedor deve salvar o estado essencial e reconstruir a tela a partir de dados persistidos.
5. `minSdk` é a versão mínima em que o app instala; `targetSdk` é a versão para a qual ele foi testado, que define os comportamentos do sistema aplicados ao app.
6. É o isolamento de cada app com identidade e dados próprios; aplica o princípio do menor privilégio.
7. Multiplataforma (Flutter ou KMP com UI nativa), pois atende às duas lojas com um código; o BLE é bem suportado por plugins e pelo KMP.
8. Resposta pessoal: Kotlin para foco em Android nativo; Flutter para entregar em várias plataformas com um código.

Fim do Módulo 00

Próximo: Módulo 01 — Ambiente de desenvolvimento, na trilha Kotlin (Android Studio e Gradle) ou na trilha Flutter (Flutter SDK e DevTools).